

From gate voltages to transition lines: how the simulator behind the dataset works

Notes for *probabilistic-csd-reconstruction*

26 September 2026

Abstract

Every device in the dataset is simulated with QArray 1.6.0 in the constant-capacitance model. This note follows one device from its 14 sampled capacitances to the two arrays the study uses: the charge-sensor map the rays are measured on, and the binary transition-line map the network is trained against. Each step is written as an equation and then re-implemented independently in `make_note.py`, which compares the result with what QArray and this repository actually produced for the stored device `sample_10`. All eight comparisons agree exactly (Table 2), so the equations below are the ones the code evaluates, not a textbook paraphrase of them. All numbers are for `sample_10`.

1 The ingredients: 14 capacitances

A device is two quantum dots (1, 2), one sensor dot (s) and three gates (g1, g2, g3). QArray takes four *non-Maxwell* matrices, whose entries are the individual capacitors of the circuit (Fig. 2(a) of the paper). For `sample_10` they are

$$\begin{aligned} C_{dd}^{\text{nm}} &= \begin{pmatrix} 1.3571 & 0.5329 \\ 0.5329 & 0.4363 \end{pmatrix}, & C_{gd}^{\text{nm}} &= \begin{pmatrix} 1.8950 & 0.4904 & 0.0943 \\ 0.0625 & 3.0132 & 0.0191 \end{pmatrix}, \\ C_{ds}^{\text{nm}} &= \begin{pmatrix} 0.0538 & 0.0487 \end{pmatrix}, & C_{gs}^{\text{nm}} &= \begin{pmatrix} 0.0718 & 0.0480 & 0.2623 \end{pmatrix}. \end{aligned} \quad (1)$$

The diagonal of C_{dd}^{nm} holds each dot's capacitance to ground (QArray's "self" capacitance, $d_1 d_1 = 1.3571$); the off-diagonal entry is the interdot capacitor ($d_1 d_2 = 0.5329$). Rows of C_{gd}^{nm} are dots and columns are gates. That is $3 + 6 + 2 + 3 = 14$ numbers. **There is no capacitance from the sensor to ground:** QArray has no argument for one, and the code passes none (Sec. 7).

2 Step 1 — the capacitance matrix

QArray's `convert_to_maxwell` turns the circuit into the total-capacitance (Maxwell) matrix C :

$$C_{ii} = \sum_j (C_{dd}^{\text{nm}})_{ij} + \sum_g (C_{gd}^{\text{nm}})_{ig}, \quad C_{ij} = -(C_{dd}^{\text{nm}})_{ij} \quad (i \neq j). \quad (2)$$

The diagonal is the *total* capacitance of a dot: everything attached to it. The off-diagonal is minus the interdot coupling.

Example. For dot 1, $C_{11} = 1.3571 + 0.5329 + 2.4797 = 4.3697$, where 2.4797 is the sum of its three gate capacitors. In full,

$$C = \begin{pmatrix} 4.3697 & -0.5329 \\ -0.5329 & 4.0640 \end{pmatrix}, \quad C^{-1} = \begin{pmatrix} 0.2326 & 0.0305 \\ 0.0305 & 0.2501 \end{pmatrix}. \quad (3)$$

$C_{11}^{-1} = 0.2326$ and $C_{22}^{-1} = 0.2501$ set the charging energy of each dot; $C_{12}^{-1} = 0.0305$ is the electrostatic coupling between them. The gate matrix used from here on is $C_g \equiv C_{gd}^{\text{nm}}$ (2×3).

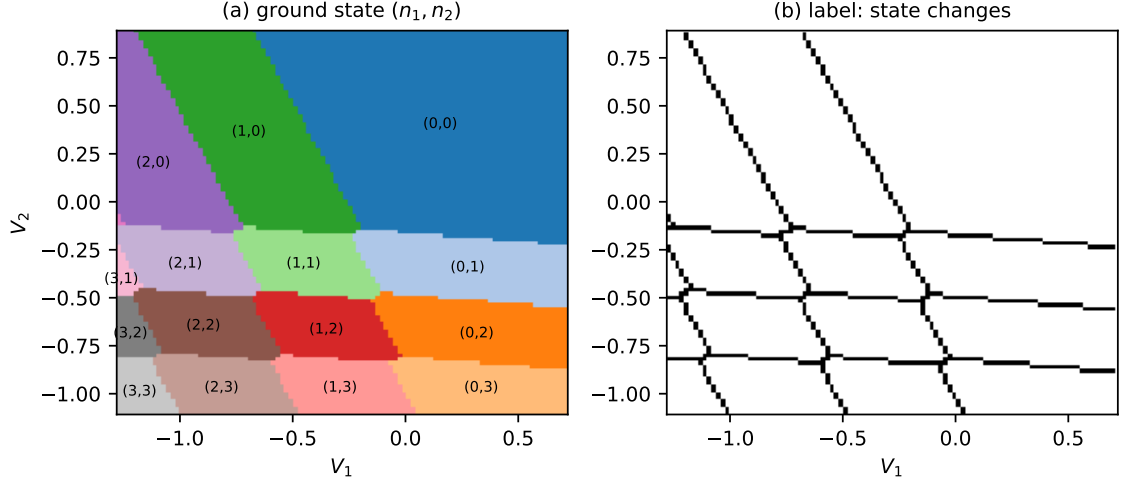


Figure 1: (a) Ground state $\mathbf{n}^*(\mathbf{V})$, Eq. (5). (b) The label, Eq. (8): every boundary in (a), one pixel wide.

3 Step 2 — the energy of a charge configuration

Let $\mathbf{n} = (n_1, n_2)$ be the number of carriers on each dot and $\mathbf{V} = (V_1, V_2, V_3)$ the gate voltages. QArray’s `free_energy` evaluates

$$F(\mathbf{n}; \mathbf{V}) = (\mathbf{n} - \mathbf{n}_g)^T C^{-1} (\mathbf{n} - \mathbf{n}_g), \quad \mathbf{n}_g = -C_g \mathbf{V} \quad (4)$$

This is the electrostatic energy of the constant-capacitance model divided by $e^2/2$; QArray works in dimensionless units throughout.

Three things are worth knowing about Eq. (4).

- $\mathbf{n}_g$ is the charge the gates *induce* on each dot. It is continuous in $\mathbf{V}$; $\mathbf{n}$ is an integer vector. The energy is lowest when $\mathbf{n}$ is as close as possible to $\mathbf{n}_g$, measured with the metric C^{-1} .
- **The minus sign is the carrier.** Both QArray models are built with holes (`charge_carrier="hole"`, the default of `ChargeSensedDotArray` as well), and for holes QArray stores $-|C_{gd}|$. So $\mathbf{n}_g$ grows as the gate voltage becomes *more negative*: in every stability diagram of this study the occupation increases towards the lower left.
- Only V_1 and V_2 are swept. V_3 is held at 0; the check “QArray V grid = ours” in Table 2 confirms it.

4 Step 3 — the ground state

At every point of the window the dots are open to a reservoir, so the occupation is whatever minimises the energy:

$$\mathbf{n}^*(\mathbf{V}) = \arg \min_{\mathbf{n} \in \mathbb{N}_0^2} F(\mathbf{n}; \mathbf{V}). \quad (5)$$

The simulation temperature is $T = 10^{-5}$ (in QArray’s units), so this is the zero-temperature ground state. `make_note.py` evaluates Eq. (5) by brute force over $n_i = 0, \dots, 5$ and obtains the same occupation as QArray at all 10000 pixels.

Worked example. At the centre pixel, $\mathbf{V} = (-0.2703, -0.0981, 0)$, which gives $\mathbf{n}_g = (0.5603, 0.3126)$. Table 1 lists F for the 16 lowest configurations. The minimum (bold) is $\mathbf{n}^* = (1, 0)$: dot 1 has 0.56 induced charges and rounds up, dot 2 has 0.31 and rounds down, and the coupling term does not change the outcome here.

Across the whole window of `sample_10` the ground state takes 16 values, $(0,0)$, $(0,1)$, $(0,2)$, $(0,3)$, $(1,0)$, $(1,1)$, $(1,2)$, $(1,3)$, $(2,0)$, $(2,1)$, $(2,2)$, $(2,3)$, $(3,0)$, $(3,1)$, $(3,2)$, $(3,3)$ (Fig. 1(a)).

Table 1: $F(\mathbf{n}; \mathbf{V})$ at the centre pixel, $\mathbf{n}_g = (0.5603, 0.3126)$.

	$n_2 = 0$	$n_2 = 1$	$n_2 = 2$	$n_2 = 3$
$n_1 = 0$	0.1081	0.1677	0.7274	1.7872
$n_1 = 1$	0.0610	0.1815	0.8022	1.9230
$n_1 = 2$	0.4790	0.6605	1.3422	2.5240
$n_1 = 3$	1.3621	1.6047	2.3473	3.5901

5 Step 4 — where the transition lines are

A transition line is where the ground state changes, i.e. where two configurations $\mathbf{n}$ and $\mathbf{n} + \mathbf{d}$ have the same energy. From Eq. (4),

$$F(\mathbf{n} + \mathbf{d}; \mathbf{V}) - F(\mathbf{n}; \mathbf{V}) = \mathbf{d}^\top C^{-1} \mathbf{d} + 2 \mathbf{d}^\top C^{-1} \mathbf{n} + 2 \mathbf{d}^\top A \mathbf{V} = 0, \quad A \equiv C^{-1} C_g. \quad (6)$$

The condition is *linear* in $\mathbf{V}$, so every boundary is a straight segment. Three kinds of step $\mathbf{d}$ occur:

$\mathbf{d}$	transition	what happens
(1, 0)	dot-1 line	one carrier enters dot 1 from the reservoir
(0, 1)	dot-2 line	one carrier enters dot 2
(1, -1)	interdot line	one carrier moves from dot 2 to dot 1

With $V_3 = 0$, Eq. (6) gives the slope of each family in the (V_1, V_2) plane:

$$\frac{dV_2}{dV_1} = - \frac{(\mathbf{d}^\top A)_1}{(\mathbf{d}^\top A)_2}, \quad A = \begin{pmatrix} 0.4426 & 0.2059 & 0.0225 \\ 0.0734 & 0.7684 & 0.0077 \end{pmatrix}. \quad (7)$$

A is the lever-arm matrix: A_{ig} is how strongly gate g moves the energy of dot i . The first two columns are what shape the diagram.

Example: slopes. Eq. (7) predicts the slopes below. To test them, `make_note.py` takes the pixels at which the ground state changes by exactly $\pm \mathbf{d}$, groups them by the pair of states on either side (each pair is one straight segment), and fits a line to the longest segment of each kind.

family	predicted, Eq. (7)	measured on the label	segment used
dot-1	-2.149	-2.148	76 points, from (0, 0)
dot-2	-0.096	-0.097	50 points, from (0, 0)
interdot	0.656	—	only 4 points

The dot lines agree to three decimals (Fig. 2). The interdot segments of this device are only a few pixels long — the interdot coupling $C_{12}^{-1} = 0.0305$ is small compared with the charging terms — so their slope cannot be measured from the pixel grid; a fit to 4 pixels returns 1.000, which says nothing.

Example: spacing. Adding one carrier to dot 1 shifts the constant in Eq. (6) by $2C_{11}^{-1}$, while moving along V_1 changes it at the rate $2A_{11}$. Successive dot-1 lines are therefore $\Delta V_1 = C_{11}^{-1}/A_{11} = 0.5254$ apart along V_1 , and dot-2 lines $\Delta V_2 = C_{22}^{-1}/A_{22} = 0.3254$ apart along V_2 — about four and a half and three lines across the 2-unit window, which is what Fig. 1 shows.

The energy along a cut. Fig. 3 follows a straight cut through the window from lower left to upper right. The top panel plots every configuration's energy above the ground state; a configuration is the ground state where its curve touches zero, and each hand-over (dotted line) is a transition. The bottom panel is the sensor signal on the same cut (Step 6): it steps at exactly those positions.

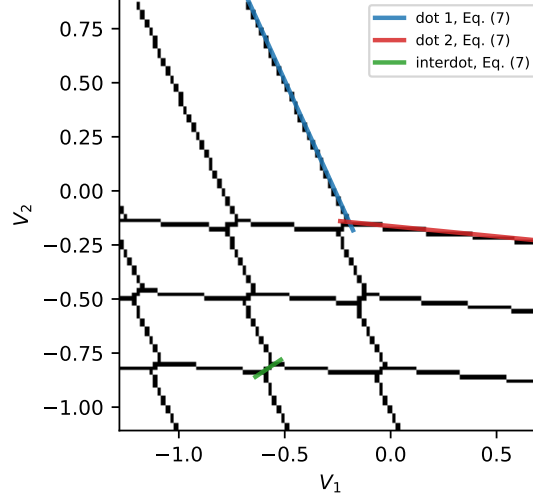


Figure 2: Slopes predicted by Eq. (7), drawn over the segments they were tested on. The interdot segment is too short to fit.

6 Step 5 — the label the network is trained on

The label is computed from the ground state, not from the sensor. In `dqd_simulator._charge_state_changes` a pixel (i, j) is marked 1 when its occupation differs from the pixel above or to the right:

$$y_{ij} = [\mathbf{n}_{i+1,j}^* \neq \mathbf{n}_{ij}^* \text{ or } \mathbf{n}_{i,j+1}^* \neq \mathbf{n}_{ij}^*]. \quad (8)$$

This gives a 99×99 array, padded with a row and a column of zeros to 100×100 . Re-computing Eq. (8) from our own ground state reproduces the stored `ground_truth_labels.npy` exactly. For `sample_10`, 5.3 % of the pixels are on a line (the dataset mean quoted in the paper is 7.1 %).

Two consequences follow directly.

- The lines are one pixel wide and sit on the boundary between states, displaced by at most one pixel from the exact degeneracy line of Eq. (6): that is the pixel quantisation the tolerance τ of the paper is designed to absorb.
- **Interdot lines are in the label** even where the sensor barely responds to them, because the label only asks whether $\mathbf{n}^*$ changed. The code comment says this is deliberate.

The label is built by a *separate* QArray model, `DotArray(Cdd, Cgd, charge_carrier="hole")`, which has no sensor. The sensor map is built by `ChargeSensedDotArray`. The two use the same dot-only C and C_g for the ground state (the sensor enters only the sensor signal), and for `sample_10` they give the same occupation at every pixel (Table 2).

7 Step 6 — the charge-sensor signal

For the sensor, QArray adds it as a third island and repeats Step 1 with 3×3 matrices. The sensor row of the non-Maxwell dot matrix is C_{ds}^{nm} and its diagonal entry is *zero*, so its total capacitance is only the sum of its couplings:

$$C_{\text{full}} = \begin{pmatrix} 4.4235 & -0.5329 & -0.0538 \\ -0.5329 & 4.1127 & -0.0487 \\ -0.0538 & -0.0487 & 0.4846 \end{pmatrix}, \quad C_{\text{full,ss}} = \sum C_{ds}^{nm} + \sum C_{gs}^{nm} = 0.4846. \quad (9)$$

With the dots held in their ground state $\mathbf{n}^*$, QArray rounds the sensor's induced charge to an integer N_s , evaluates the full energy $F_k = F_{\text{full}}((\mathbf{n}^*, N_s + k); \mathbf{V})$ for $k = -5, \dots, 5$, and sums a Lorentzian of width $\gamma = 0.01$ over the energy steps:

$$z(\mathbf{V}) = \sum_{k=-5}^{5-1} \frac{1}{1 + [(F_{k+1} - F_k)/\gamma]^2}. \quad (10)$$

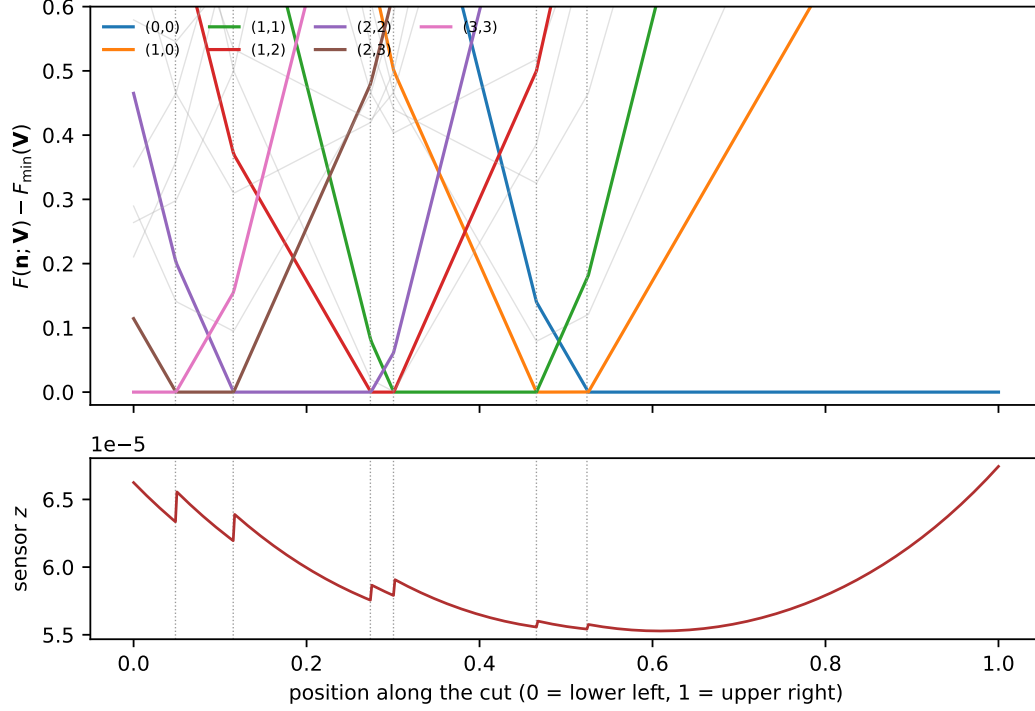


Figure 3: Top: $F(\mathbf{n}; \mathbf{V}) - \min_{\mathbf{n}} F$ along a diagonal cut, for every configuration within 0.6 of the ground state; the ones that become the ground state are coloured. Bottom: sensor signal z , Eq. (10), on the same cut. Dotted lines: where the ground state changes.

$F_{k+1} - F_k$ is the energy to add one carrier to the sensor; a Coulomb peak is where it vanishes. When a dot's occupation changes, the off-diagonal terms of C_{full}^{-1} shift that energy, and z jumps: the steps in Figs. 3 and 4. Eq. (10) re-implemented reproduces QArray's `charge_sensor_open` at every pixel (largest difference: exactly zero).

Where the sensor sits. Over the window of `sample_10`, z lies between 5.53×10^{-5} and 7.18×10^{-5} of the peak height 1, and the smallest $|F_{k+1} - F_k|$ anywhere is 1.421, i.e. 142γ . The sensor is therefore never near a Coulomb peak: it operates deep in the tail of the Lorentzian, where z falls off roughly as $\gamma^2/(F_{k+1} - F_k)^2$. An experimental sensor is usually tuned to the flank of a peak, where the response is largest. The steps are still exact and noise-free here, so the reconstruction task is not affected; but it is a difference from an experiment worth knowing when the method is transferred. This has been checked for `sample_10` only.

8 Checks

Every equation above was re-implemented without calling QArray's functions and compared with QArray and with the stored files of `sample_10`.

Table 2: Independent re-implementation against QArray and the stored data, `sample_10`.

comparison	result
C matches QArray	pass
C.full matches QArray	pass
hole sign: <code>cgd = -C.g</code>	pass
ground state = QArray <code>DotArray</code> (all pixels)	pass
ground state = stored <code>charge.states.npy</code>	pass
label = stored ground truth (interior)	pass
sensor signal = QArray (all pixels)	pass
QArray V grid = ours ($V_3 = 0$)	pass

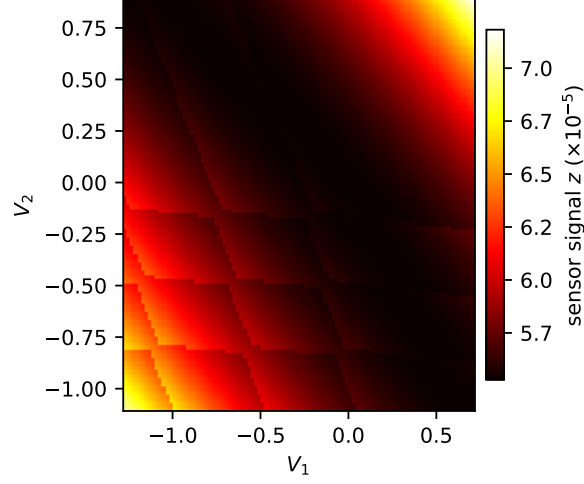


Figure 4: Sensor signal z of sample_10 from Eq. (10); identical to QArray’s output. The smooth background is the Lorentzian tail; the honeycomb is the steps.

9 What this means for the paper

- The sensor map and the label come from the same ground state, so the lines the network is asked to draw are exactly the ones that produce the steps in the sensor signal — plus the interdot lines, which the label contains even where the sensor hardly sees them.
- Every line is straight, with a slope fixed by the lever-arm matrix A ; the honeycomb spacing is fixed by C_{ii}^{-1}/A_{ii} . Both vary from device to device because all 14 capacitances are drawn independently (Table I of the paper).
- The simulation is at zero temperature and noise-free (NoNoise); the only broadening is the Coulomb-peak width γ of the sensor.
- The sensor dot has no capacitance to ground, and in sample_10 it sits far from any Coulomb peak. Neither affects where the lines are, which is what the study measures.

Reproducing this note.

```
python docs/simulation_physics/make_note.py
cd docs/simulation_physics && latexmk -pdf simulation_physics.tex
```

The script reads the device from data/_device_pools/devices_n550_res100_c1df7b6bf/sample_10.